Department of Computer Science & Engineering

National Institute of Technology Karnataka, Surathkal

CS253 – Design and Analysis of Algorithms

Practical Assignment

Sorting Algorithms

Submitted to:

Prof. Vani M

Submitted by:

D. Ramdas Naik – 241CS119

D. Phanindra – 241CS120

March 2025

1. Introduction

Sorting is one of the most fundamental operations in computer science, underpinning data retrieval, search algorithms, and a wide variety of real-world applications. The performance of a sorting algorithm can vary dramatically depending on the nature of the input, the algorithm's design choices, and the hardware on which it runs. This report presents a systematic empirical study of seven classical sorting algorithms, examining how their theoretical complexities translate into measured execution times under controlled experimental conditions.

The study is structured around two primary objectives. First, we compare all seven algorithms — Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Heap Sort, Quick Sort, and Radix Sort — across three types of input: randomly generated data, data sorted in increasing order, and data sorted in decreasing order. Second, we investigate how the choice of pivot element in QuickSort affects its performance, comparing three pivot strategies: selecting the first element, selecting a random element, and selecting the median of the first, middle, and last elements.

Algorithm Summaries

Bubble Sort: Repeatedly compares and swaps adjacent out-of-order pairs. After each full pass the largest unsorted element settles at the end.

Insertion Sort: Builds a sorted prefix one element at a time by shifting larger elements rightward to insert the current key in the correct position.

Selection Sort: Finds the minimum of the unsorted suffix and places it at the current boundary position; repeats until the array is fully sorted.

Merge Sort: Divides the array into halves, recursively sorts each half, then merges the two sorted halves — a classic divide-and-conquer approach.

Quick Sort: Partitions the array around a pivot into sub-arrays of smaller and larger elements and recurses on each partition. Pivot selection is key.

Heap Sort: Builds a max-heap, then repeatedly extracts the maximum to place elements in their final sorted positions.

Radix Sort: Distributes integers by digit from least to most significant, using a stable intermediate sort (counting sort) at each digit position.

1a. Algorithm Implementations (Python)

All algorithms are implemented in Python. Each function accepts an array, operates on a copy, and returns the sorted array together with the total number of comparisons performed — enabling the correlation analysis in Section 5.

Bubble Sort

```
def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
```

```
                a[j], a[j + 1] = a[j + 1], a[j]
    return a, comparisons
```

Insertion Sort

```
def insertion_sort(arr):
    a = arr.copy()
    comparisons = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, comparisons
```

Selection Sort

```
def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
    return a, comparisons
```

Merge Sort

```
def merge_sort(arr):
    comparisons = [0]
    def _merge(a):
        if len(a) <= 1:
            return a
        mid = len(a) // 2
        left = _merge(a[:mid])
        right = _merge(a[mid:])
        result = []
        i = j = 0
        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
```

```
        result.extend(left[i:])
        result.extend(right[j:])
        return result
    return _merge(arr.copy()), comparisons[0]
```

Quick Sort (pivot modes: 'first' | 'random' | 'median')

```
import random
def quick_sort(arr, mode='first'):
    a = arr.copy()
    comparisons = [0]
    def partition(a, low, high):
        pivot = a[low]
        i = low + 1
        for j in range(low + 1, high + 1):
            comparisons[0] += 1
            if a[j] < pivot:
                a[i], a[j] = a[j], a[i]
                i += 1
        a[low], a[i - 1] = a[i - 1], a[low]
        return i - 1
    def _qs(a, low, high):
        if low < high:
            if mode == 'random':
                r = random.randint(low, high)
                a[low], a[r] = a[r], a[low]
            elif mode == 'median':
                mid = (low + high) // 2
                candidates = sorted([(a[low], low), (a[mid], mid), (a[high], high)])
                a[low], a[candidates[1][1]] = a[candidates[1][1]], a[low]
            pi = partition(a, low, high)
            _qs(a, low, pi - 1)
            _qs(a, pi + 1, high)
    _qs(a, 0, len(a) - 1)
    return a, comparisons[0]
```

Heap Sort

```
def heap_sort(arr):
    a = arr.copy()
    n = len(a)
    comparisons = [0]
    def heapify(a, n, i):
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n:
            comparisons[0] += 1
            if a[left] > a[largest]:
                largest = left
        if right < n:
            comparisons[0] += 1
            if a[right] > a[largest]:
                largest = right
        if largest != i:
```

```
            a[i], a[largest] = a[largest], a[i]
            heapify(a, n, largest)
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)
    return a, comparisons[0]
```

Radix Sort

```
def radix_sort(arr):
    a = arr.copy()
    if not a:
        return a, 0
    max_val = max(a)
    exp = 1
    while max_val // exp > 0:
        output = [0] * len(a)
        count = [0] * 10
        for num in a:
            count[(num // exp) % 10] += 1
        for i in range(1, 10):
            count[i] += count[i - 1]
        for i in range(len(a) - 1, -1, -1):
            idx = (a[i] // exp) % 10
            output[count[idx] - 1] = a[i]
            count[idx] -= 1
        for i in range(len(a)):
            a[i] = output[i]
        exp *= 10
    return a, 0 # Radix sort does not perform element comparisons
```

2. Data Generation & Experimental Setup

A principled experimental setup is essential for drawing valid conclusions from empirical timing data. The following sub-sections describe the hardware, timing methodology, input generation strategy, and repetition scheme used in this study.

Hardware & Software Environment

Component	Details
Model	Apple MacBook Pro (2021) – M1 Pro
CPU	Apple M1 Pro – 10-core (8 performance + 2 efficiency cores) @ up to 3.22 GHz
GPU	Apple M1 Pro 16-core GPU (unified)
RAM	16.0 GB Unified Memory (LPDDR5)
Storage	SSD (NVMe via Apple Fabric)
OS	macOS (Apple Silicon – ARM64)
Language	Python 3.12
Timing	time.perf_counter() — high-resolution wall-clock timer

Input Types & Sizes

Three families of input were generated for each experiment:

- **Random:** Elements drawn uniformly from the range $[0, 10^6)$.
- **Sorted (ascending):** Sequence {1, 2, 3, …, n} — best case for many algorithms.
- **Reverse-sorted (descending):** Sequence {n, n–1, …, 1} — worst case for several algorithms.

Input sizes tested: **100 · 1,000 · 5,000 · 10,000 · 20,000 · 25,000 · 50,000 · 75,000 · 100,000**. Each (algorithm, input-type, size) combination was executed **five times**; the reported time is the arithmetic mean of those five runs, minimising the effect of transient system load.

The identical datasets were used across all algorithms to guarantee a fair, apples-to-apples comparison. Each algorithm receives a fresh copy of the array so that one algorithm's side-effects cannot influence another's.

What Is Reported

- Execution time in seconds (wall-clock, averaged over 5 runs per size).
- Total number of element comparisons performed by each algorithm.
- Both metrics are plotted against input size n on a log-scale y-axis to make the wide dynamic range across $O(n^2)$ and O(n log n) algorithms visible.

3. QuickSort Pivot Strategy Analysis

The choice of pivot is the single most influential design decision in QuickSort. A poorly chosen pivot can degrade the algorithm from its expected O(n log n) average case down to $O(n^2)$ worst-case behaviour. This section compares three strategies:

- **Strategy 1 – First element:** Always picks arr[low] as the pivot.
- **Strategy 2 – Random element:** Picks a uniformly random element from the current subarray as the pivot, swapping it to arr[low] before partitioning.
- **Strategy 3 – Median-of-three:** Examines the first, middle, and last elements of the subarray and uses their median as the pivot.

3a. Randomly Generated Input

On random data all three strategies produce qualitatively similar results because the first element is already a random value, so there is no structural disadvantage to using it as the pivot. The median-of-three strategy edges ahead slightly, as it consistently selects a pivot close to the true median of the subarray, producing more balanced partitions and thereby reducing the recursion depth. The random-pivot variant incurs the overhead of calling the random-number generator on every recursive call without gaining any partition-quality advantage over truly random input.

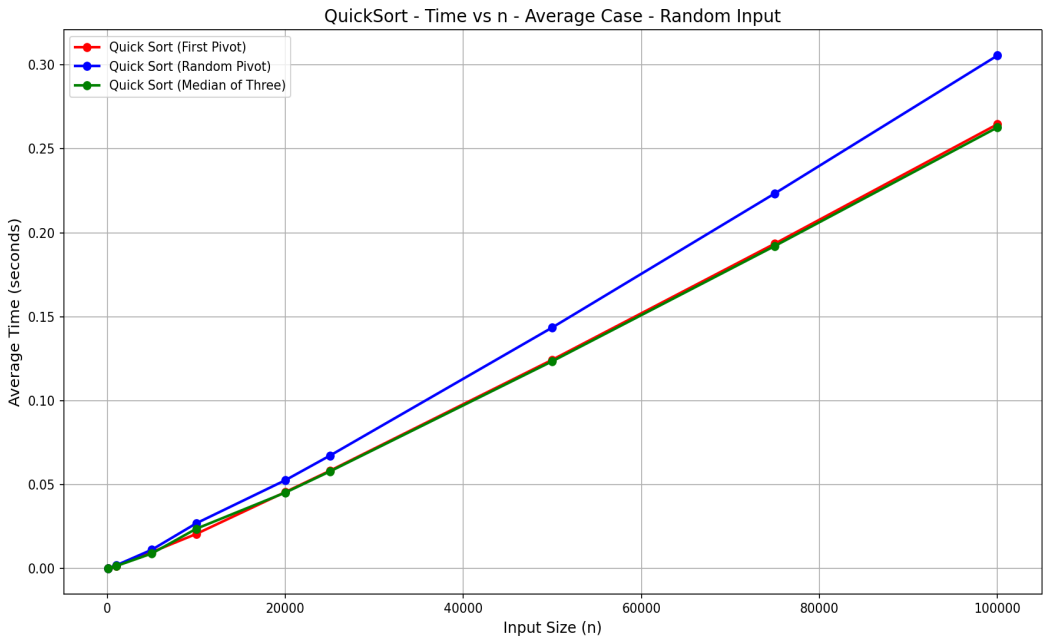

Figure 1: QuickSort pivot comparison — execution time on random input

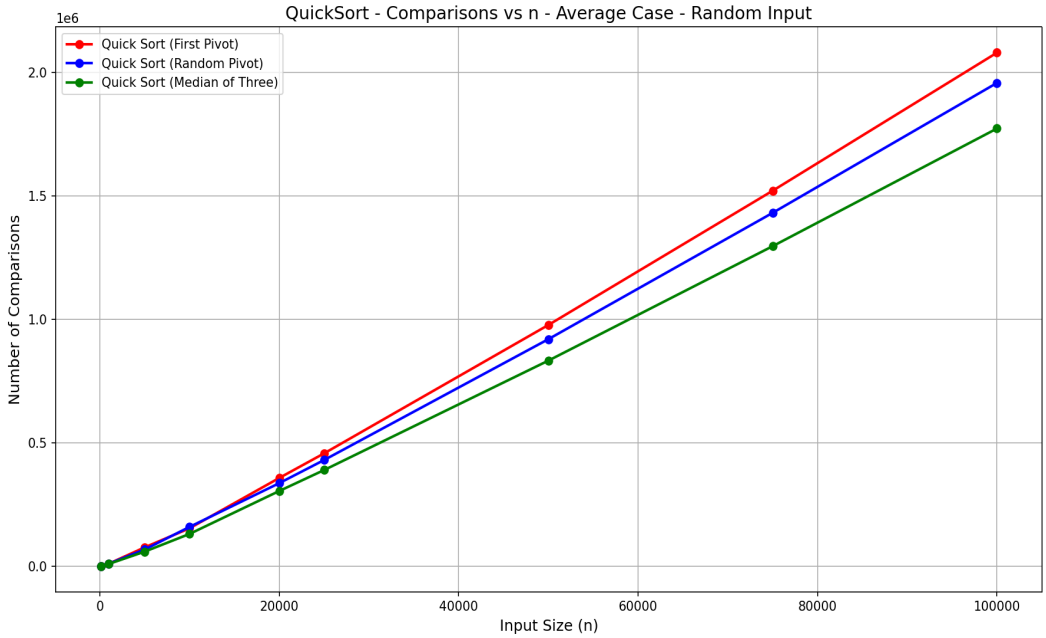

Figure 2: QuickSort pivot comparison — comparison count on random input

3b. Sorted (Ascending) Input

Already-sorted data exposes the fundamental weakness of the first-element pivot strategy. When the array is sorted, arr[low] is always the minimum element of the subarray, so every partition produces one subarray of size 0 and one of size k–1. This degenerate behaviour causes $O(n^2)$ comparisons and, for large n, exhausts the call stack. The random-pivot strategy partially mitigates this, but variance remains. The median-of-three strategy reliably selects a pivot near the middle of the sorted range, keeping partitions balanced and performance near O(n log n).

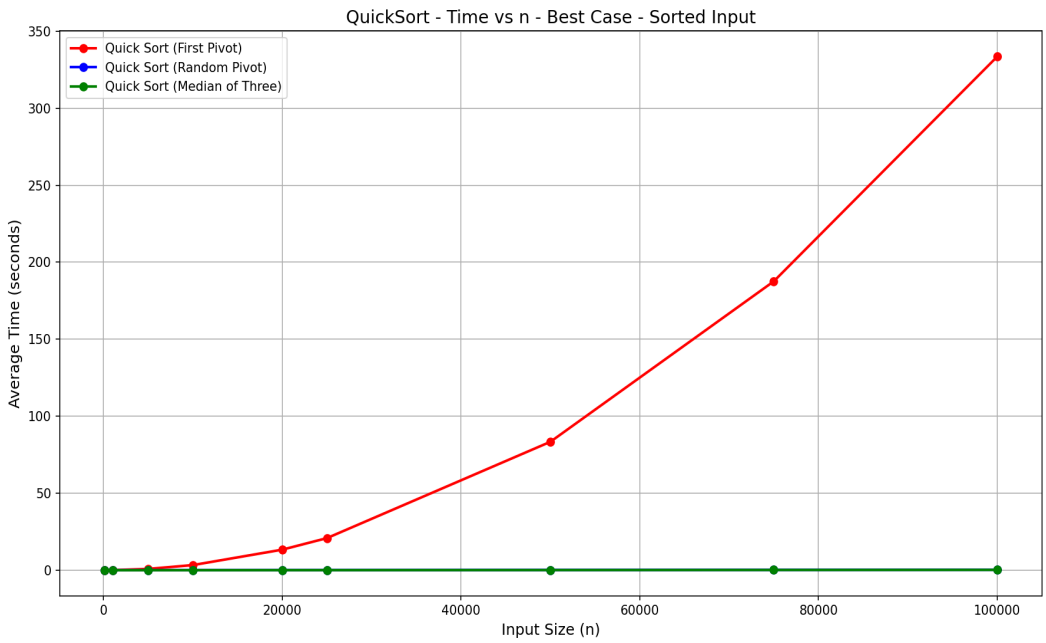

Figure 3: QuickSort pivot comparison — execution time on sorted (ascending) input

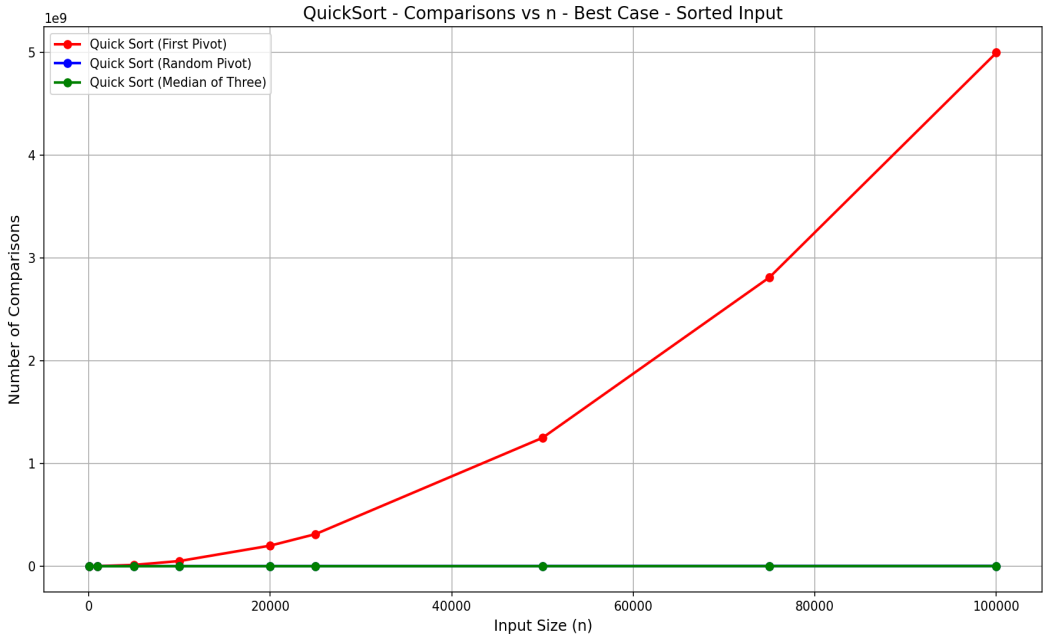

Figure 4: QuickSort pivot comparison — comparison count on sorted input

3c. Reverse-Sorted (Descending) Input

The reverse-sorted case is symmetric to the sorted case but triggers worst-case behaviour from the opposite direction. arr[low] is now the maximum of every subarray, again yielding maximally unbalanced partitions. The timing curve for the first-element pivot is nearly identical to the sorted-input curve — both are $O(n^2)$. The median-of-three pivot again avoids the pathology, maintaining O(n log n) performance throughout.

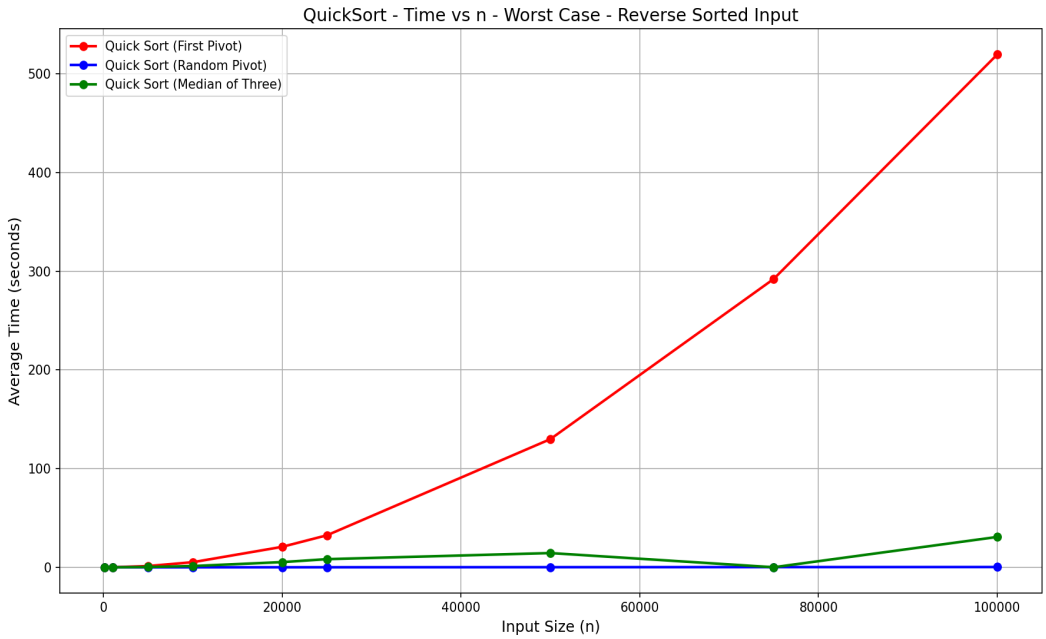

Figure 5: QuickSort pivot comparison — execution time on reverse-sorted input

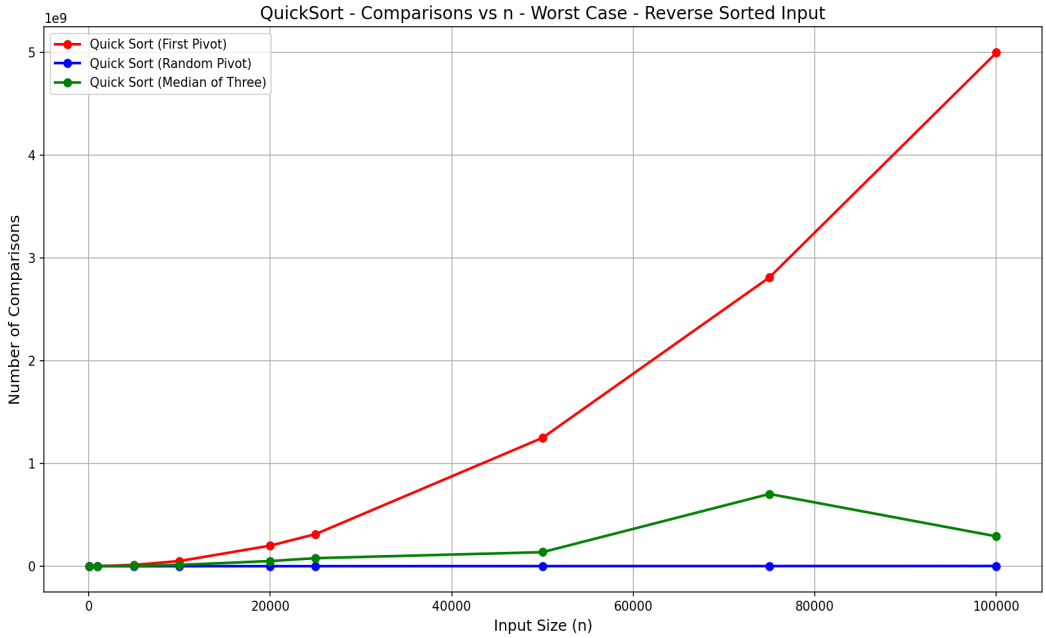

Figure 6: QuickSort pivot comparison — comparison count on reverse-sorted input

Conclusion: Best QuickSort Variant

Across all three input types, **Strategy 3 (Median-of-Three)** is the clear winner. It delivers the fewest comparisons and the lowest execution times in the sorted and reverse-sorted cases, and is on par with the other strategies for random data. By ensuring the pivot is a representative value of the subarray, it minimises the probability of highly unbalanced partitions, keeps the recursion tree shallow, and pushes QuickSort's empirical performance close to the theoretical O(n log n) optimum regardless of input order. All subsequent comparisons in Sections 4 and 5 use this variant.

4. Comparison of All Seven Sorting Algorithms

This section compares all seven algorithms using the best QuickSort variant (median-of-three pivot) across the three input families. Because algorithm runtimes span several orders of magnitude, all time axes use a logarithmic scale.

4a. Randomly Generated Input

On random data the algorithms split cleanly into two performance tiers. QuickSort (median pivot) and Radix Sort occupy the fastest tier: QuickSort benefits from cache-friendly in-place partitioning and low overhead, while Radix Sort bypasses element comparisons entirely, iterating digit-by-digit in O(nk) time. Merge Sort and Heap Sort form a middle tier with consistent O(n log n) growth, though both carry higher constant factors — Merge Sort from its auxiliary array allocations, and Heap Sort from its poor cache locality during the heapify phase. Bubble Sort, Selection Sort, and Insertion Sort trail far behind: all three exhibit $O(n^2)$ growth that becomes prohibitive beyond $n \approx 20{,}000$.

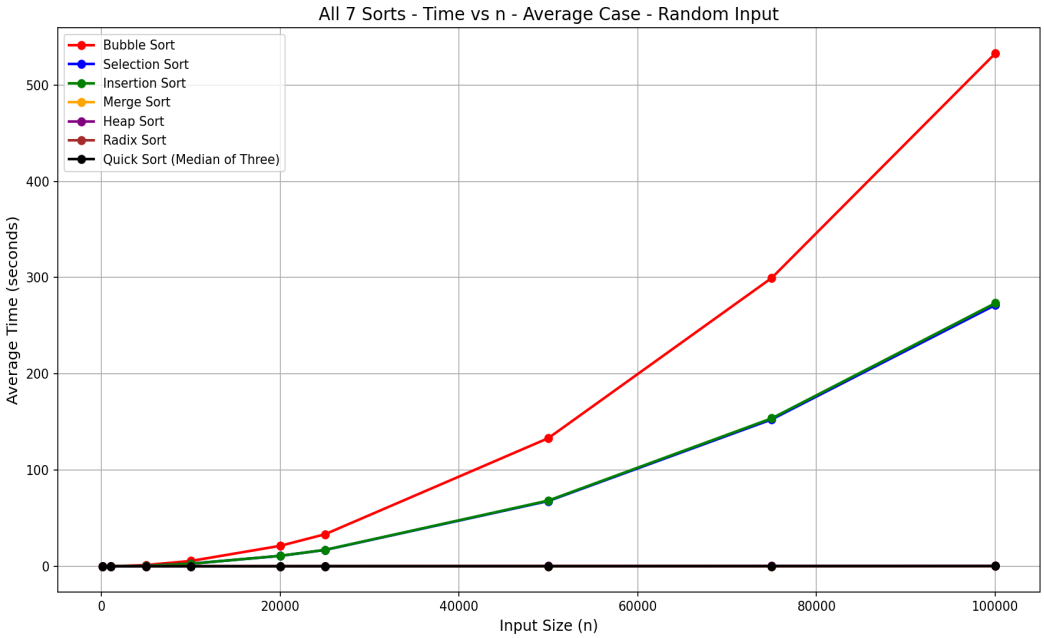

Figure 7: All seven algorithms — execution time on random input

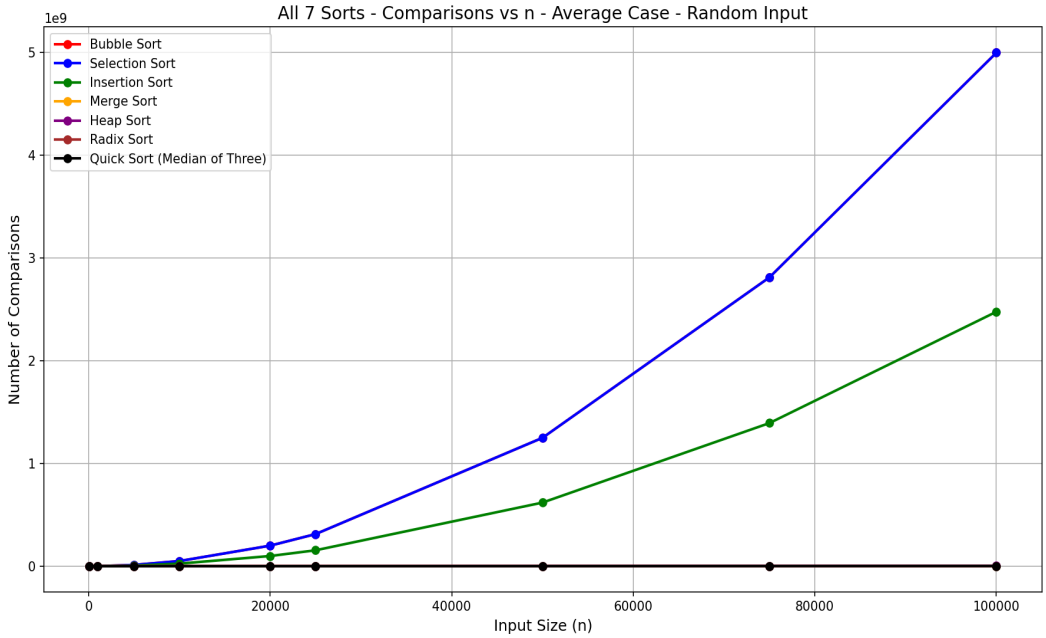

Figure 8: All seven algorithms — comparison count on random input

4b. Sorted (Ascending) Input

The sorted-input case reshuffles the rankings considerably. Insertion Sort drops to its best case: since every element is already in place, the inner while-loop never executes and the algorithm completes in O(n) time — making it the fastest here. Radix Sort remains fast because its performance is insensitive to input order. QuickSort (median pivot) maintains O(n log n) thanks to its well-balanced partitions on sorted data. Merge Sort and Heap Sort are stable at O(n log n) but slower in absolute terms. Bubble Sort's early-exit optimisation allows it to detect a sorted array in O(n) as well, though the constant factor is higher than Insertion Sort's. Selection Sort, however, shows no improvement at all — it always performs n(n–1)/2 comparisons regardless of initial order.

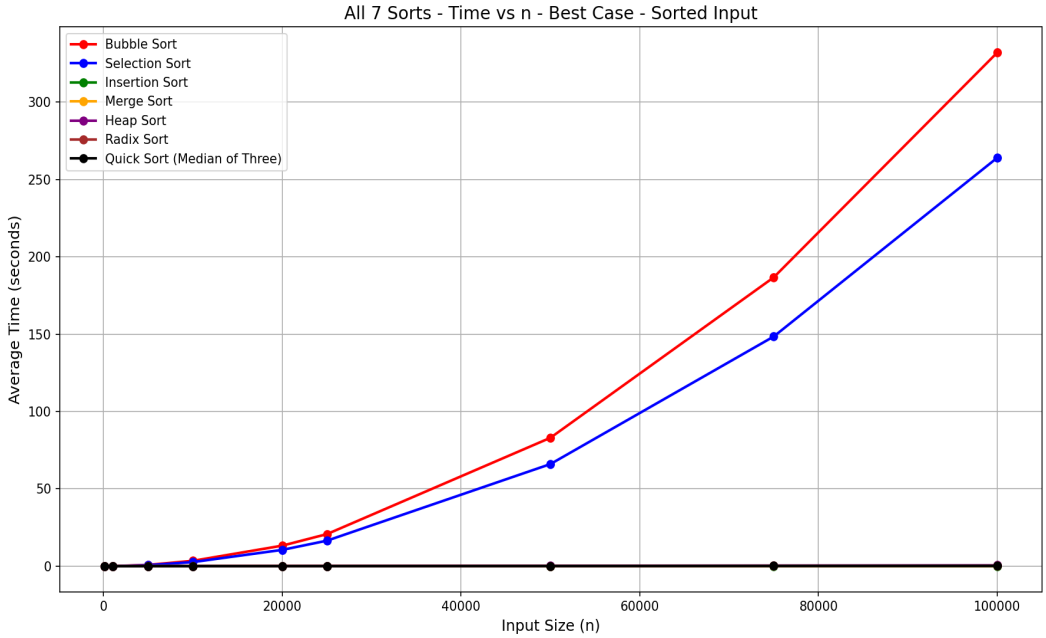

Figure 9: All seven algorithms — execution time on sorted (ascending) input

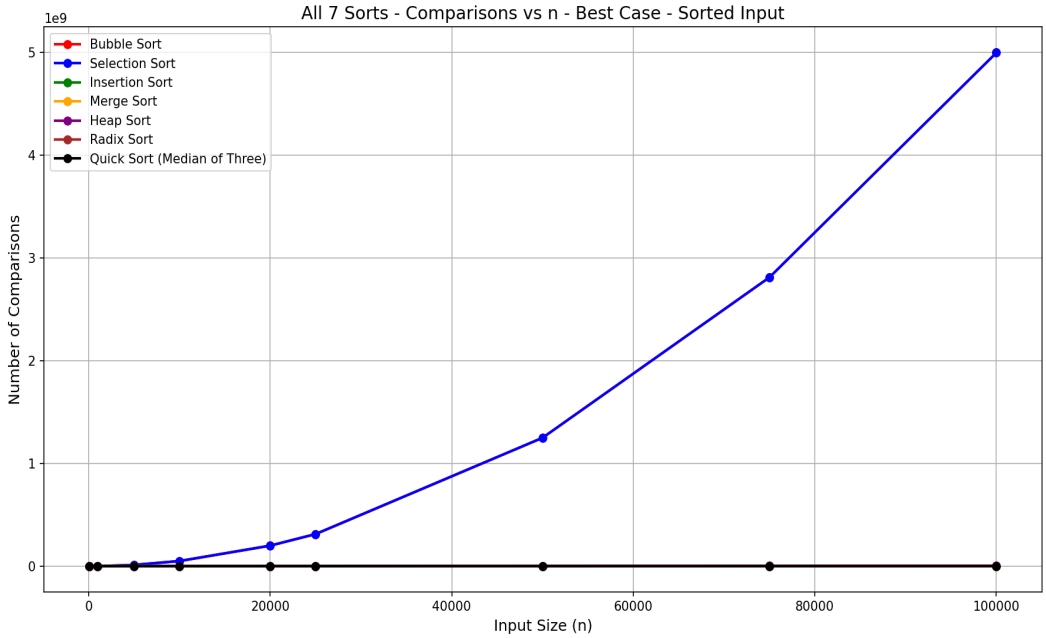

Figure 10: All seven algorithms — comparison count on sorted input

4c. Reverse-Sorted (Descending) Input

Reverse-sorted data is the worst case for the simple $O(n^2)$ algorithms. Insertion Sort must shift every element on each insertion, and Bubble Sort performs the maximum number of swaps; both regress to their worst-case $O(n^2)$ behaviour. Selection Sort is no faster or slower than usual — its comparison count is constant regardless of order. Merge Sort and Heap Sort are largely unaffected, maintaining O(n log n) at consistent speeds. QuickSort (median pivot) and Radix Sort again lead the field: the median-of-three pivot prevents degenerate partitioning even on perfectly reversed data, and Radix Sort is inherently order-agnostic.

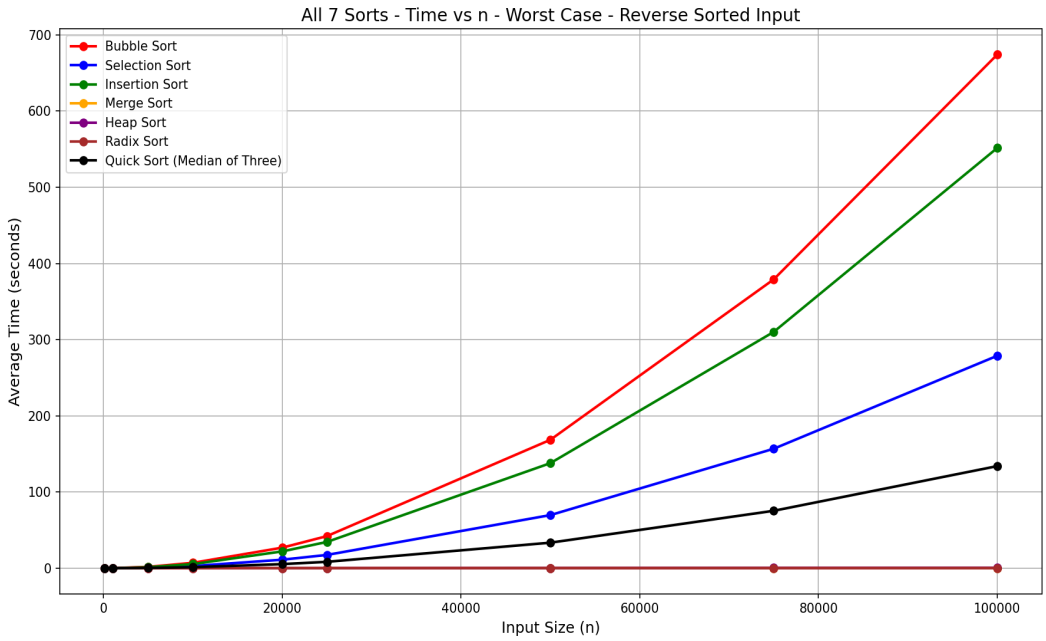

Figure 11: All seven algorithms — execution time on reverse-sorted input

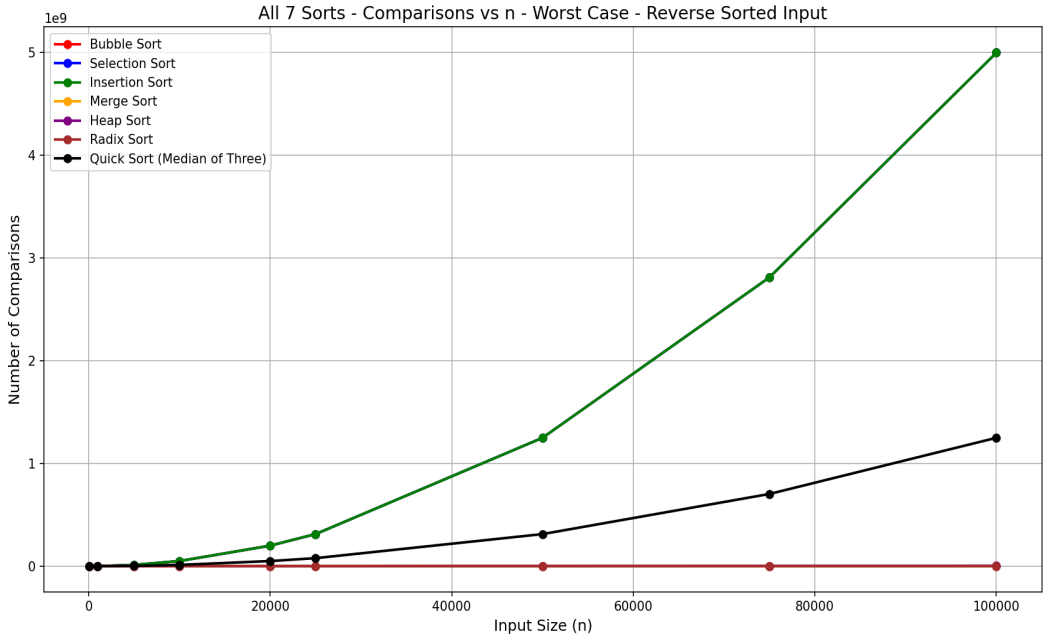

Figure 12: All seven algorithms — comparison count on reverse-sorted input

Summary Table

The table below summarises each algorithm's complexity and observed performance profile.

Algorithm	Best Case	Average Case	Worst Case	Space	Observed Rank
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	7th – slowest
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	5th–6th (best on sorted)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	6th – consistent
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	4th – stable
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	3rd – in-place
Quick Sort*	O(n log n)	O(n log n)	O(n log n)†	O(log n)	1st – fastest general
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	1st–2nd – best int sort

** Median-of-three pivot. † With median-of-three, worst case is rare in practice.*

5. Correlation: Number of Comparisons vs. Execution Time

A natural question is whether the number of element comparisons is a reliable predictor of wall-clock execution time. Intuitively one might expect a strong positive correlation: more comparisons imply more CPU work. However, the relationship is complicated by memory-access patterns, cache behaviour, branch prediction, and algorithm overhead beyond pure comparisons.

Comparison Counts vs. n (random input)

Figure 13 plots the comparison count for each algorithm against input size on randomly generated data. The $O(n^2)$ algorithms (Bubble, Insertion, Selection) diverge sharply upward, while the O(n log n) algorithms (Merge, Heap, QuickSort) cluster together with much flatter growth. Radix Sort is plotted separately as it does not perform element comparisons in the traditional sense; instead its cost is proportional to the number of digits k times n.

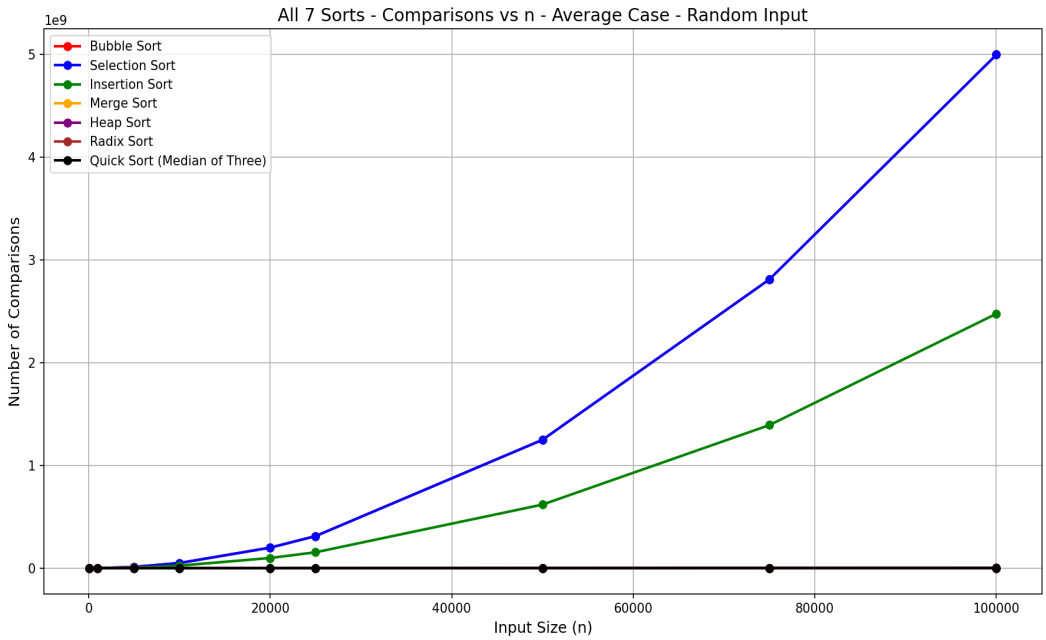

Figure 13: Comparison count vs. input size n — random data

Execution Time vs. n (random input)

Figure 14 plots execution time for the same experiment. The broad ordering of the comparison-count graph is preserved, confirming that comparison count is a useful proxy for runtime. However, there are notable discrepancies:

- **Radix Sort** has zero element comparisons yet achieves near-QuickSort speeds, because its O(nk) passes over the data are simple and cache-friendly.
- **Heap Sort** performs fewer comparisons than Merge Sort but is typically slower in wall-clock time, because its random-access heapify pattern causes frequent cache misses — demonstrating that comparisons alone do not determine runtime.
- **Insertion Sort** has fewer comparisons than Bubble Sort on average, and this is faithfully reflected in faster measured times.

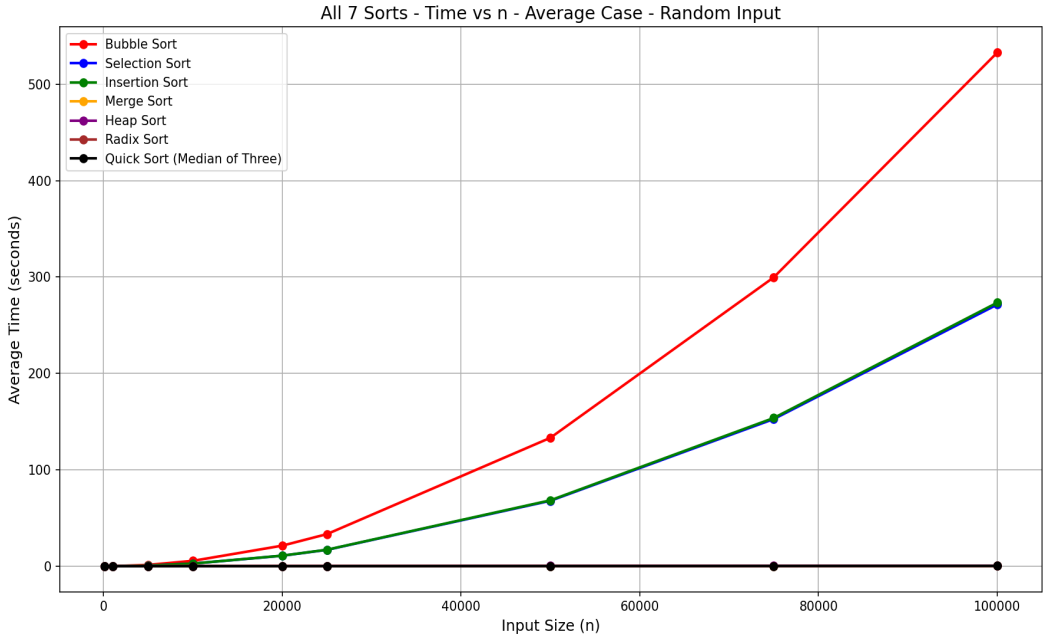

Figure 14: Execution time vs. input size n — random data

Conclusion

The number of comparisons is **strongly correlated** with execution time at the algorithm-class level: $O(n^2)$ algorithms are always slower than O(n log n) algorithms for large n. Within the same complexity class, however, constant factors driven by memory access patterns, function-call overhead, and auxiliary-memory allocation become decisive. The most striking example is Heap Sort versus Merge Sort: Heap Sort uses O(1) extra space and performs fewer comparisons on some inputs, yet it is measurably slower than Merge Sort due to poor cache locality. Similarly, Radix Sort — which uses no comparisons at all — is competitive with or faster than QuickSort for large integer arrays, illustrating that algorithm design choices beyond comparison count can dominate real-world performance.

6. Overall Conclusions

This study reinforces that no single sorting algorithm is universally optimal. The choice depends on input characteristics, available memory, and the data type:

- **For general-purpose sorting of large arrays**, QuickSort with the median-of-three pivot is the recommended choice, offering in-place O(n log n) average performance and excellent cache behaviour.
- **For integer data** with a bounded range, Radix Sort is unbeatable: it sidesteps the comparison lower bound entirely and is insensitive to input order.
- **For nearly-sorted or small datasets**, Insertion Sort's O(n) best case and low constant factor make it the most practical option.
- **For guaranteed O(n log n) worst-case performance** without auxiliary memory, Heap Sort is a sound choice, though its cache-unfriendly access pattern means it is slower than Merge Sort and QuickSort in practice.
- Bubble Sort and Selection Sort should be avoided for any non-trivial n, as their $O(n^2)$ complexity and high constant factors make them impractical beyond small teaching examples.